SaTTric+ Reconfigurable Rule Engine — Protocol & Firmware Design

1. Design goal

Nothing in SaTTric+ firmware should be hardcoded to a specific energy meter, sensor range, or communication port. Instead, firmware implements four generic **engines** (Modbus, I/O, UART, Output/Event), and a remote UI writes **configuration records** that tell each engine what to actually do. Reconfiguring the device — new meter brand, new sensor range, new alarm threshold — is a data change pushed over the wire, never a firmware reflash.

2. Transport-agnostic frame format

Field	Size	Notes
STX	1 byte	0x02, frame start marker
SEQ	1 byte	Rolling sequence number, detects drops/replays
CMD	1 byte	Opcode (see Section 3)
LEN	2 bytes	Payload length, little-endian
PAYLOAD	0–240 bytes	TLV-encoded config record(s) — capped by a 256-byte device receive buffer
CRC16	2 bytes	CCITT CRC16 over SEQ..PAYLOAD
ETX	1 byte	0x03, frame end marker

Responses reuse the same layout with `CMD | 0x80`; payload byte 0 is a status code (`0x00 OK`, `0x01 ERR_CRC`, `0x02 ERR_RANGE`, `0x03 ERR_BUSY`, `0x04 ERR_AUTH`, `0x05 ERR_UNKNOWN_CMD`).

This frame is carried as-is over a direct UART commissioning link, and identically base64/binary-wrapped inside an MQTT payload or HTTPS body for remote/cloud configuration — the firmware-side parser is transport-agnostic. Remote (cloud) sessions add an authentication token and TLS at the transport layer; the core frame itself stays unchanged so the same parser and validation logic serve both paths.

3. Command set

Note on authority: this document describes the command set at a conceptual level. The companion Hardware Bridge Protocol Spec (now at v1.1) is the byte-accurate source of truth for exact payload layouts, and has moved ahead of this document with several additions closing gaps against the global device spec — Ethernet/Modbus TCP support, an on-device audit log, remote carbon-factor profile management, device health reporting, and signed firmware OTA. Where the two documents differ on a detail, the Bridge Protocol Spec wins.

Opcode	Name	Purpose
0x01	GET_DEVICE_INFO	Firmware version, hardware ID, config version
0x02	GET_DEVICE_HEALTH	CPU/RAM/flash health, uptime, link status *(v1.1)*
0x03	SET_DEVICE_IDENTITY	Secure onboarding/provisioning *(v1.1, architecture-level only — see Bridge Protocol Spec 6.14)*
0x10	SET_METER_MAP	Write a Modbus register-map template for a meter, over RTU **or** TCP *(TCP added in v1.1)*
0x11	GET_METER_MAP	Read back current register-map template
0x12	SET_SECONDARY_MODBUS	Configure the auxiliary Modbus bus/device
0x13	SET_CARBON_FACTOR_PROFILE	Write a carbon-factor profile with validity window *(v1.1)*
0x14	GET_CARBON_FACTOR_PROFILE	Read back a carbon-factor profile *(v1.1)*
0x20	SET_IO_CHANNEL_CONFIG	Configure one universal I/O channel, including digital polarity *(polarity added in v1.1)*
0x21	GET_IO_CHANNEL_CONFIG	Read back I/O channel configuration
0x22	SET_IO_CALIBRATION	Push two-point calibration for a channel
0x30	SET_OUTPUT_CONFIG	Configure a relay or PWM output, including its control authority *(now 5 modes — see Section 4 below)*
0x40	SET_UART_CONFIG	Configure baud/mode/role for a UART port
0x50	SET_EVENT_RULE	Define a threshold → action rule
0x51	GET_EVENT_RULE	Read back a rule
0x53-0x55	SET_OUTPUT_STATE / GET_OUTPUT_STATE / AI_HEARTBEAT	Immediate actuation for remote-supervisory or AI-driven outputs, with actor

Opcode	Name	Purpose
		attribution *(v1.1)*
0x56	GET_AUDIT_LOG	Read the device's own tamper-resistant event record *(v1.1)*
0x60	COMMIT_CONFIG	Atomically activate staged configuration
0x61	ROLLBACK_CONFIG	Revert to last committed configuration
0x62-0x65	Firmware update commands	Chunked, signed OTA image transfer *(v1.1)*
0x70	GET_LIVE_TAG_VALUE	Read current value from the Tag Database

All `SET_*` **configuration** commands stage changes into a shadow config area; nothing takes effect on the live engines until `COMMIT_CONFIG` succeeds. This makes multi-record config pushes atomic — the device never runs on a half-applied configuration. The actuation and firmware-update commands added in v1.1 are the exception, by design — see the Bridge Protocol Spec Section 5 for which commands stage versus act immediately.

4. Config record structures

4.1 Meter register-map entry (one per measured parameter)

Field	Type	Description
param_id	uint16	Internal tag ID this maps to (e.g. 0x0001 = Voltage_L1)
modbus_function	uint8	0x03 (holding) / 0x04 (input) etc.
start_register	uint16	Register address on the target meter
reg_count	uint8	Registers to read (1, 2, or 4)
data_type	uint8	0=INT16 1=UINT16 2=INT32 3=UINT32 4=FLOAT32 5=BCD
byte_order	uint8	0=big-endian 1=little-endian 2=word-swapped
scale_factor	float32	Raw value × scale_factor
offset	float32	Added after scaling
poll_interval_ms	uint16	How often this register is polled

A meter register map is simply an array of these entries. Switching from a Schneider meter to a Secure or Rishabh meter is loading a different array — the polling engine itself doesn't change.

This is the mechanism that lets SaTTric+ work with “any Modbus energy meter” as described in the product brief: the meter’s register layout becomes data, not code.

4.2 Device/bus descriptor (primary meter bus and secondary Modbus bus)

Field	Type	Description
bus_id	uint8	0 = primary meter bus, 1 = secondary/aux bus
slave_address	uint8	Modbus slave ID
baud	uint32	e.g. 9600, 19200
parity	uint8	0=none 1=even 2=odd
stop_bits	uint8	1 or 2
response_timeout_ms	uint16	Per-transaction timeout
retry_count	uint8	Retries before flagging comms fault
register_map_id	uint16	Points to a stored register-map template (Section 4.1)

4.3 I/O channel config (one per universal I/O channel, 0–7)

Field	Type	Description
channel_id	uint8	0–7
mode	uint8	0=ANALOG_0_10V 1=ANALOG_4_20MA 2=DIGITAL_10_30V 3=PULSE
tag_id	uint16	Where scaled value is written in the Tag Database
raw_min / raw_max	uint16	ADC counts at the two calibration points
eng_min / eng_max	float32	Corresponding engineering-unit values
unit_id	uint8	Enum: bar, °C, %RH, l/min, etc., or custom string table index
filter_ms	uint16	Debounce (digital) or smoothing window (analog)

4.4 Two-point calibration record

Field	Type	Description
channel_id	uint8	Target channel
raw_point_1, eng_point_1	uint16, float32	First calibration pair
raw_point_2, eng_point_2	uint16, float32	Second calibration pair

Firmware computes gain/offset from the two points at commit time: `gain = (eng2 - eng1) / (raw2 - raw1)`, `offset = eng1 - gain × raw1`.

4.5 Output config (relay 0–7 or PWM 0–3)

Field	Type	Description
output_id	uint8	Relay 0–7 or PWM 0–3 (type implied by ID range)
control_mode	uint8	0=MANUAL 1=RULE_DRIVEN 2=SCHEDULE 3=REMOTE_SUPERVISORY 4=AI_SERVER_DRIVEN *(v1.1 — see below)*
linked_rule_id	uint16	Event rule that drives this output (if RULE_DRIVEN)
pwm_frequency_hz	uint16	PWM channels only
default_duty_pct	uint8	PWM channels only, 0–100
fail_safe_state	uint8	State to force on comms-loss/watchdog event
authority_timeout_ms	uint16	*(v1.1)* Watchdog window for modes 3/4 — see below

Three control authorities, not two. As of v1.1, `control_mode` distinguishes a local rule engine, an authenticated remote *person* (`REMOTE_SUPERVISORY`), and the server's AI decision loop (`AI_SERVER_DRIVEN`) as three separate authorities — not two, and not one generic "remote" bucket. This matters for the product's audit positioning: every live command sent to a REMOTE_SUPERVISORY or AI_SERVER_DRIVEN output carries an `actor_type` and `actor_id` (Bridge Protocol Spec Section 6.10), and every accepted command writes an entry to the device's own audit log (Section 4.7 below), so "was this a person or the AI" is a fact recorded on the device at the moment it happened — not something reconstructed after the fact from server-side records that sit upstream of the device and could be incomplete.

`authority_timeout_ms` governs both remote authorities: if a `REMOTE_SUPERVISORY` or `AI_SERVER_DRIVEN` output doesn't receive a command or heartbeat within this window, the device forces it to `fail_safe_state` on its own, with no dependency on the network recovering. Ignored (send 0) for modes 0-2.

4.6 Carbon-factor profile *(v1.1)*

Field	Type	Description
profile_id	uint16	Device stores up to 16 profiles, addressed by ID
factor_value	float32	kg CO2e per kWh
valid_from	uint32	Unix epoch seconds
valid_until	uint32	Unix epoch seconds; open-

Field	Type	Description
		ended sentinel value if ongoing
region_code	uint16	Grid-region identifier

This is what makes tags 43-45 (EmissionsFactor_Value/ProfileID/ValidityWindow) remotely *writable*, not just readable — closing the "remote carbon-factor profile update with validity-window management and audit history" requirement from the global device spec. Every committed change to an active profile writes an audit log entry recording the old and new factor values.

4.7 Audit log entry *(v1.1, device-generated — not something the UI writes)*

Field	Type	Description
timestamp	uint32	Unix epoch seconds
actor_type	uint8	LOCAL_RULE / MANUAL / REMOTE_SUPERVISORY / AI_SERVER / SYSTEM
actor_id	uint32	Matches the actor_id from the triggering command, where applicable
event_type	uint8	OUTPUT_STATE_CHANGE / CONFIG_COMMIT / CONFIG_ROLLBACK / CALIBRATION_CHANGE / CARBON_FACTOR_UPDATE / FIRMWARE_UPDATE / DEVICE_PROVISIONED / AUTH_FAILURE
target_id	uint16	output_id, channel_id, rule_id, or profile_id, depending on event_type
old_value, new_value	float32, float32	Before/after values

Stored in a dedicated circular flash partition, separate from the config dual-bank, and retrieved via `GET_AUDIT_LOG`. This is the record a compliance or audit review would actually pull from — see Bridge Protocol Spec Section 6.11 for exact retrieval mechanics.

4.8 UART port config (applies to any of the four UART ports)

Field	Type	Description
port_id	uint8	0=meter bus 1=aux bus 2=GPRS 3=spare/diagnostic
baud, parity, stop_bits	—	Same as Section 4.2
protocol_mode	uint8	0=MODBUS_RTU 1=RAW_PASSTHROUGH

Field	Type	Description
		2=SENSOR_PROTOCOL
role_locked	bool	Prevents remote reassignment of safety-critical ports

Making UART mode a config field (rather than compile-time) is what lets a port be repurposed for a non-Modbus sensor protocol without a firmware rebuild — exactly the "UART configurable as per sensor" requirement.

4.9 Event rule (threshold → action)

Field	Type	Description
rule_id	uint16	Unique ID
source_tag_id	uint16	Tag Database entry this rule watches
operator	uint8	0=GT 1=LT 2=GTE 3=LTE 4=EQ 5=NEQ
threshold	float32	Comparison value
hysteresis	float32	Prevents relay chatter near the threshold
action_output_id	uint8	Output this rule drives
action_type	uint8	0=SET 1=CLEAR 2=TOGGLE 3=SET_PWM_DUTY
action_value	float32	Used for SET_PWM_DUTY

5. Firmware execution model

1. **Modbus poll task** — for each configured bus (primary + secondary), walks its register-map template on the configured `poll_interval_ms` per entry, applies `scale_factor/offset`, and writes the result into the Tag Database with a timestamp.
2. **I/O scan task** — samples all 8 universal channels per their configured mode, applies the calibration gain/offset from Section 4.4, and writes scaled engineering values into the Tag Database.
3. **Pulse/UART sensor task** — services pulse counters and any UART port configured in `SENSOR_PROTOCOL` mode, writing totals/readings into the Tag Database.
4. **Rule evaluation task** — runs whenever a watched tag updates; evaluates the `operator/threshold/hysteresis` and drives the linked output through the Output/Event engine.
5. **Comm task** — parses incoming frames, validates CRC/sequence, stages `SET_*` commands into shadow config, and only swaps in the new configuration on `COMMIT_CONFIG`.

6. **Telemetry task** — pushes Tag Database deltas to the cloud/GPRS link on a configurable interval, independent of the local rule evaluation cadence.
7. **Actuation authority task** *(v1.1)* — handles `SET_OUTPUT_STATE/AI_HEARTBEAT` for outputs in `REMOTE_SUPERVISORY` or `AI_SERVER_DRIVEN` mode, enforces that the command's `actor_type` matches the output's configured authority, resets the per-output watchdog on every accepted command or heartbeat, and forces `fail_safe_state` if `authority_timeout_ms` elapses without either.
8. **Audit task** *(v1.1)* — writes one entry to the circular audit log for every accepted output-state change, config commit/rollback, calibration change, carbon-factor update, firmware update, and provisioning event or auth failure. Runs synchronously with the triggering event rather than batched, so the log ordering is trustworthy even across a power loss immediately after the triggering action.

6. Configuration storage and safety

- Configuration is stored in **dual-bank SPI flash** (active bank + shadow bank). A `SET_*` sequence writes to the shadow bank; `COMMIT_CONFIG` verifies a checksum over the shadow bank, then flips a single "active bank" pointer — this makes the commit atomic even if power is lost mid-write, since the device always boots from whichever bank the pointer currently designates as valid.
- `ROLLBACK_CONFIG` simply flips the pointer back without needing to resend data.
- If the active bank ever fails its checksum at boot (corruption, aborted write), the device falls back to a factory-default configuration held in a separate protected flash region, and flags a config-fault status so the UI knows to re-push configuration.
- Every commit increments a `config_version` counter, returned by `GET_DEVICE_INFO`, so the remote UI can detect drift between what it last pushed and what the device is actually running.

7. Remote (cloud) access considerations

- The core frame (Section 2) is unaware of transport; a commissioning laptop on the spare UART and a cloud UI over GPRS/MQTT use the identical command set and payload structures.
- Remote sessions add a bearer token validated by the Comm task before any `SET_*` or `COMMIT_CONFIG` is accepted — local-UART sessions can optionally skip this for factory commissioning, but that is a build-time policy flag, not a protocol difference.
- The `SEQ` byte combined with a monotonic session nonce (carried at the transport layer) prevents replay of captured configuration frames.
- `role_locked` on UART config (Section 4.6) and similar lock flags on safety-relevant outputs prevent a remote misconfiguration from disabling the ports needed to recover the device (e.g. the meter bus or the diagnostic UART).

8. UI-side representation

The UI/cloud tool is not expected to hand-encode binary TLV records. A practical pattern: the UI authors configuration as JSON (one object per Section 4 structure), and a thin translation layer — either in the cloud backend or in a UI-side SDK — packs it into the binary frame

format before it reaches the device. This keeps the constrained serial/GPRS link efficient while keeping the authoring experience human-readable.